

Algoritmos de aproximación avanzados

Redondeo, Vertex Cover, PTAS y FPTAS

Docente: Héctor Eduardo Ugarte Rojas

Universidad Nacional de San Antonio Abad del Cusco
Carrera Profesional de Ingeniería Informática y de Sistemas

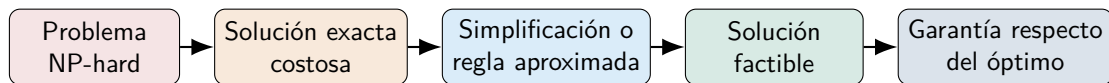
Semestre académico 2026-II

Propósito y resultados esperados

Propósito: estudiar técnicas que permiten controlar formalmente la pérdida de calidad al simplificar problemas de optimización difíciles.

Al finalizar la sesión, el estudiante podrá:

- 1 explicar el papel del redondeo en el diseño de aproximaciones;
- 2 ejecutar el algoritmo 2-aproximado para Minimum Vertex Cover;
- 3 demostrar la garantía de dicho algoritmo mediante un matching maximal;
- 4 distinguir aproximación constante, PTAS y FPTAS;
- 5 interpretar el parámetro de precisión ε ;
- 6 relacionar Programación Dinámica pseudo-polinomial y FPTAS.



Pregunta guía

¿Cómo podemos acercarnos arbitrariamente al óptimo sin ejecutar necesariamente un algoritmo exacto?

Recordatorio: garantía multiplicativa

Para un problema de **minimización**, una ρ -aproximación satisface:

$$\text{ALG}(I) \leq \rho \text{ OPT}(I), \quad \rho \geq 1.$$

Para un problema de **maximización** una convención frecuente es:

$$\text{ALG}(I) \geq \frac{1}{\rho} \text{ OPT}(I), \quad \rho \geq 1.$$

Interpretación

La garantía se demuestra para **toda instancia válida**. La razón observada $r(I)$ describe solamente una instancia concreta.

¿Qué significa redondear una instancia?

Redondear consiste en reemplazar valores por otros más estructurados para reducir la complejidad de la instancia.

- Reducir la cantidad de valores distintos.
- Agrupar elementos en clases o niveles.
- Reducir la cantidad de estados de una Programación Dinámica.
- Resolver la instancia transformada.
- Traducir la solución a la instancia original.
- Demostrar una cota sobre el error introducido.

Principio

El redondeo no es arbitrario: su granularidad controla el intercambio entre tiempo de ejecución y calidad.

Ejemplo básico de redondeo

Supongamos los valores:

37, 42, 58, 61.

Al redondear al múltiplo de 10 más cercano:

40, 40, 60, 60.

Ventaja

La instancia tiene ahora solo dos valores distintos y puede resumirse mediante sus frecuencias.

Riesgo

La solución óptima de la instancia redondeada puede no ser óptima para la instancia original.

Redondeo hacia arriba o hacia abajo

Para comparar ambas operaciones, considere:

$$x = 37, \quad K = 10.$$

Hacia arriba

$$\hat{x} = K \left\lceil \frac{x}{K} \right\rceil = 10 \lceil 3.7 \rceil = 40.$$

Se selecciona el siguiente múltiplo de 10.

Aplicación: un archivo de 37 MB requiere 40 MB si el almacenamiento se reserva en bloques de 10 MB.

$$37 \leq 40 < 37 + 10.$$

Hacia abajo

$$\tilde{x} = K \left\lfloor \frac{x}{K} \right\rfloor = 10 \lfloor 3.7 \rfloor = 30.$$

Se selecciona el múltiplo de 10 anterior.

Aplicación: un beneficio de 37 puede aproximarse por 30 para reducir los estados de una Programación Dinámica.

$$37 - 10 < 30 \leq 37.$$

Interpretación

El redondeo hacia arriba sobreestima en 3 unidades; el redondeo hacia abajo subestima en 7. En ambos casos, el error es menor que $K = 10$.

Patrón de análisis del redondeo

- 1 Definir la transformación y el tamaño de escala K .
- 2 Relacionar el óptimo de la instancia original con el de la instancia redondeada.
- 3 Resolver eficientemente la instancia simplificada.
- 4 Reconstruir una solución factible para la instancia original.
- 5 Acotar la pérdida total introducida por el redondeo.

Forma típica

pérdida por elemento $\times$ cantidad de elementos $\leq \varepsilon \text{ OPT}$.

Patrón de análisis del redondeo

Ejemplo numérico

Supongamos una instancia de Knapsack con: (numero de objetos, mayor valor individual, valor solución optima, tolerancia de aproximación)

$$n = 5, \quad V_{\text{máx}} = 80, \quad \text{OPT} = 100, \quad \varepsilon = 0.10.$$

La escala de redondeo es:

$$K = \frac{\varepsilon V_{\text{máx}}}{n} = \frac{0.10(80)}{5} = 1.6.$$

Como cada elemento pierde menos de K :

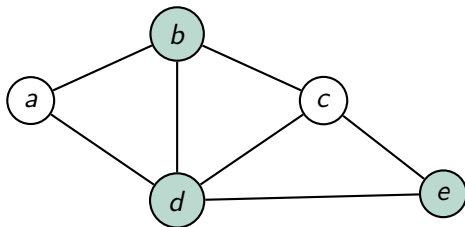
$$\underbrace{1.6}_{\text{pérdida por elemento}} \cdot \underbrace{5}_{\text{cantidad de elementos}} = 8 \leq 10 = \underbrace{0.10(100)}_{\varepsilon \text{ OPT}}.$$

La pérdida total es, por tanto, menor que el 10 % del óptimo.

Minimum Vertex Cover

Dado un grafo no dirigido $G = (V, E)$, se busca un conjunto $C \subseteq V$ de cardinalidad mínima tal que:

$$\forall (u, v) \in E, \quad u \in C \text{ o } v \in C.$$



En este grafo, $\{b, d, e\}$ es una cobertura; cada arista tiene al menos un extremo seleccionado.

Algoritmo 2-aproximado para Vertex Cover

Procedimiento

- ❶ Inicializar $C \leftarrow \emptyset$.
- ❷ Mientras exista una arista no eliminada (u, v) :
 - ❶ agregar u y v a C ;
 - ❷ eliminar todas las aristas incidentes en u o en v .
- ❸ Retornar C .

Estructura generada

Las aristas elegidas no comparten extremos. Por tanto, forman un **matching maximal** M .

Ejecución paso a paso

Considere el camino:

$$a - b - c - d - e.$$

- 1 Elegir (a, b) : agregar a, b y eliminar (a, b) y (b, c) .
- 2 Quedan (c, d) y (d, e) .
- 3 Elegir (c, d) : agregar c, d y eliminar ambas aristas restantes.
- 4 Retornar:

$$C = \{a, b, c, d\}, \quad |C| = 4.$$

Una cobertura óptima es:

$$C^* = \{b, d\}, \quad |C^*| = 2.$$

En esta ejecución se alcanza exactamente la razón 2 (es justamente el doble, es una 2-aproximación).

¿Por qué la salida siempre es una cobertura?

Supongamos que al terminar queda una arista (x, y) no cubierta.
Entonces:

- ni x ni y pertenecen a C ;
- la arista (x, y) no fue eliminada por ser incidente en un vértice seleccionado;
- por tanto, el algoritmo habría podido seguir seleccionando (x, y) .

Esto contradice que el algoritmo haya terminado.

Conclusión

La salida C es siempre una solución factible de Vertex Cover.

¿Por qué es una 2-aproximación?

Sea M el conjunto de aristas elegidas por el algoritmo.

- ① Ninguna pareja de aristas de M comparte un extremo.
- ② Cualquier cobertura debe tomar al menos un extremo de cada arista de M .
- ③ Por tanto:

$$|M| \leq \text{OPT}.$$

- ④ El algoritmo agrega los dos extremos de cada arista:

$$|C| = 2|M|.$$

En consecuencia:

$$|C| = 2|M| \leq 2 \text{OPT}.$$

Matching maximal no significa matching máximo

Un matching, en teoría de grafos, es un conjunto de aristas que no comparten vértices entre sí.

Matching maximal

No puede añadirse otra arista sin compartir un extremo con alguna ya elegida. Es una condición local.

Matching máximo

Es un matching con la mayor cantidad posible de aristas. Es una condición global.

Importante

El algoritmo 2-aproximado solo necesita construir un matching maximal, que puede obtenerse vorazmente (greedy). No necesita calcular un matching máximo.

Con listas de adyacencia o una lista de aristas y marcas apropiadas, el algoritmo puede implementarse en:

$$\mathcal{O}(|V| + |E|).$$

- Evitar eliminaciones físicas costosas si basta marcar aristas o vértices.
- Mantener el grafo original para verificar la salida.
- Comprobar al final:

$$\forall (u, v) \in E, \quad u \in C \vee v \in C.$$

- El orden de elección puede cambiar C , pero no elimina la garantía 2.

Calidad observada en Vertex Cover

Si se conoce una cobertura óptima C^* , la razón observada es:

$$r(I) = \frac{|C|}{|C^*|}.$$

Para el camino de cinco vértices:

$$|C| = 4, \quad |C^*| = 2,$$

por lo que:

$$r(I) = \frac{4}{2} = 2.$$

No generalizar desde un caso

Otra elección de aristas puede producir una cobertura diferente. La demostración, no el ejemplo, establece que la razón nunca supera 2.

¿Qué es un esquema de aproximación?

Un esquema recibe la instancia y un parámetro de precisión $\varepsilon > 0$.
Para minimización, la meta habitual es:

$$\text{ALG}_\varepsilon(I) \leq (1 + \varepsilon) \text{OPT}(I).$$

ε grande

Menor exigencia de calidad y, normalmente, menor tiempo de ejecución.

ε pequeño

Mayor cercanía al óptimo y, normalmente, mayor tiempo de ejecución.

PTAS: Polynomial-Time Approximation Scheme

Un PTAS es una familia de algoritmos tal que, para cada $\varepsilon > 0$ fijo:

- produce una solución con garantía $(1 + \varepsilon)$ en minimización;
- se ejecuta en tiempo polinomial respecto del tamaño n de la entrada.

El grado del polinomio puede depender de $1/\varepsilon$.

Tiempo permitido en un PTAS

$$T(n, \varepsilon) = n^{1/\varepsilon}.$$

Para ε fijo es polinomial en n , aunque puede resultar impracticable cuando ε es pequeño.

¿Por qué $n^{1/\varepsilon}$ es polinomial para ε fijo?

$$T(n, \varepsilon) = n^{1/\varepsilon}.$$

ε fijo

Si elegimos:

$$\varepsilon = 0.10, \quad \frac{1}{\varepsilon} = 10,$$

entonces:

$$T(n, 0.10) = n^{10}.$$

El exponente 10 es constante. Por tanto, el tiempo es **polinomial en n** .

$$\varepsilon \text{ fijo} \Rightarrow n^{1/\varepsilon} = n^c$$

ε puede variar

Si la precisión depende de n , por ejemplo:

$$\varepsilon = \frac{1}{n}, \quad \frac{1}{\varepsilon} = n,$$

entonces:

$$T\left(n, \frac{1}{n}\right) = n^n.$$

El exponente crece con n . Por tanto, el tiempo **no es polinomial en n** .

$$\varepsilon = \frac{1}{n} \Rightarrow n^{1/\varepsilon} = n^n$$

FPTAS: Fully Polynomial-Time Approximation Scheme

Un FPTAS también produce una solución arbitrariamente cercana al óptimo, pero exige que el tiempo sea polinomial en:

$$n \quad \text{y} \quad \frac{1}{\varepsilon}.$$

Tiempo permitido en un FPTAS

$$T(n, \varepsilon) = \mathcal{O}\left(\frac{n^3}{\varepsilon^2}\right).$$

Relación

Todo FPTAS es un PTAS, pero no todo PTAS es un FPTAS.

¿Por qué un FPTAS es totalmente polinomial?

Un FPTAS debe tener tiempo polinomial simultáneamente en, con una forma permitida:

$$n \quad \text{y} \quad \frac{1}{\varepsilon}.$$

$$T(n, \varepsilon) = \frac{n^3}{\varepsilon^2} = n^3 \left(\frac{1}{\varepsilon} \right)^2.$$

$$\varepsilon = 0.10$$

Como:

$$\frac{1}{\varepsilon} = 10,$$

se obtiene:

$$T(n, 0.10) = n^3(10)^2 = 100n^3.$$

La precisión introduce un factor constante; el exponente de n sigue siendo 3.

$$\varepsilon = 1/n$$

Como:

$$\frac{1}{\varepsilon} = n,$$

se obtiene:

$$T\left(n, \frac{1}{n}\right) = n^3 n^2 = n^5.$$

Resultado polinomial con exponentes constantes.

PTAS frente a FPTAS

Característica	PTAS	FPTAS
Calidad	$(1 + \varepsilon)$	$(1 + \varepsilon)$
Polinomial en n	Sí, para ε fijo	Sí
Polinomial en $1/\varepsilon$	No necesariamente	Sí
Ejemplo de tiempo	$n^{1/\varepsilon}$	n^3/ε^2
Relación	Clase más amplia	Todo FPTAS es PTAS

En el contexto de PTAS y FPTAS, normalmente se prefieren valores:

$$0 < \varepsilon < 1$$

Clasificar por la forma del tiempo

Suponga que todos producen la calidad requerida.

Caso	Tiempo	Clasificación
A	$n^{2/\varepsilon}$	PTAS; no necesariamente FPTAS
B	n^4/ε^3	FPTAS
C	$2^{1/\varepsilon} n^3$	PTAS; no necesariamente FPTAS
D	$2^n/\varepsilon$	No es PTAS

Razón

En un PTAS, para cada ε fijo el tiempo debe ser polinomial en n . En un FPTAS también debe ser polinomial en $1/\varepsilon$.

De pseudo-polinomial a FPTAS

Knapsack 0/1 tiene una Programación Dinámica que puede formularse en función de la suma de valores:

$$T(n, V) = \mathcal{O}(nV).$$

Si V (suma de valores de objetos) es grande, el algoritmo es pseudo-polinomial, es decir su tiempo es polinomial en el valor numérico de ciertos datos, pero no necesariamente en la cantidad de bits utilizados para representarlos. n : es el numero de objetos. Una estrategia de FPTAS es:

- 1 elegir una escala K dependiente de ε ;
- 2 reducir los valores mediante redondeo;
- 3 ejecutar la DP sobre los valores escalados;
- 4 reconstruir la selección original;
- 5 demostrar que la pérdida total es pequeña.

Escalamiento de valores en Knapsack

Sea $V_{\text{máx}} = \max_i v_i$ y:

$$K = \frac{\varepsilon V_{\text{máx}}}{n}, \quad v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

siempre que el peso del objeto i sea menor o igual que la capacidad W . i : índice del objeto
Entonces:

$$v'_i \leq \frac{n}{\varepsilon} \implies \sum_i v'_i \leq \frac{n^2}{\varepsilon}.$$

v'_i es el valor escalado del objeto i

Cada objeto pierde menos de K unidades de valor. Para una selección de, como máximo, n objetos:

$$\text{pérdida total} < nK = \varepsilon V_{\text{máx}} \leq \varepsilon \text{OPT}.$$

Ejemplo: cálculo de $V_{\text{máx}}$

Sea una mochila con capacidad $W = 10$:

i	w_i	v_i	Estado
1	4	30	Factible
2	7	80	Factible
3	12	200	Se descarta: $w_3 > W$
4	5	60	Factible

El valor 200 no se considera porque el objeto 3 no cabe individualmente. Por tanto:

$$V_{\text{máx}} = \max\{v_i : w_i \leq W\} = \max\{30, 80, 60\} = 80.$$

¿Por qué $V_{\text{máx}} \leq \text{OPT}$?

El objeto de valor 80 cabe por sí solo y constituye una solución factible. Como OPT es la mejor solución factible:

$$V_{\text{máx}} = 80 \leq \text{OPT}.$$

Ejemplo: cálculo de K y escalamiento

Después de descartar el objeto 3 quedan tres objetos factibles:

$$n = 3, \quad V_{\text{máx}} = 80.$$

Si elegimos:

$$\varepsilon = 0.10,$$

la escala de redondeo es:

$$K = \frac{\varepsilon V_{\text{máx}}}{n} = \frac{0.10(80)}{3} = \frac{8}{3} \approx 2.67.$$

Los valores escalados se calculan mediante:

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

Ejemplo: cálculo de K y escalamiento

Objeto	v_i	v_i/K	$v'_i = \lfloor v_i/K \rfloor$
1	30	11.25	11
2	80	30.00	30
4	60	22.50	22

Resultado

La Programación Dinámica utiliza los valores escalados

(11, 30, 22)

en lugar de los valores originales (30, 80, 60).

La selección obtenida se evalúa finalmente con los valores originales, no con los valores escalados.

Garantía resultante para Knapsack

Si S^* es una solución óptima y S es la solución recuperada mediante los valores escalados, se obtiene una garantía de la forma:

$$V(S) \geq (1 - \varepsilon) \text{OPT}.$$

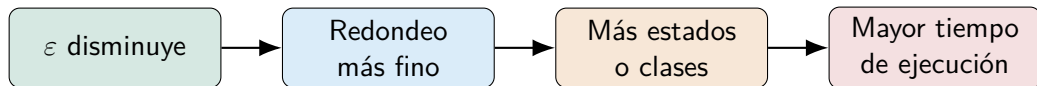
Interpretación

Con $\varepsilon = 0.1$, se garantiza al menos el 90 % del valor óptimo bajo esta convención.

Cuidado con las convenciones

En maximización también puede expresarse una garantía como $V(S) \geq \text{OPT} / (1 + \varepsilon)$. Siempre debe declararse la desigualdad exacta utilizada.

¿Qué ocurre al disminuir ε ?



Al mismo tiempo

La garantía se acerca a 1 en minimización o la fracción garantizada se acerca a 1 en maximización.

Qué no demuestra la experimentación

- Encontrar el óptimo en muchas pruebas no demuestra una garantía universal.
- La peor razón observada no es automáticamente el factor de aproximación.
- Un gráfico no sustituye las demostraciones de factibilidad y calidad.
- Los experimentos sí permiten estudiar comportamiento típico, constantes, memoria y escalabilidad.

teoría: toda instancia	$\neq$	experimento: instancias evaluadas
------------------------	--------	-----------------------------------

Actividad 1: Vertex Cover

Para el camino $a - b - c - d - e$:

- 1 ejecute el algoritmo seleccionando primero (a, b) ;
- 2 ejecútelo nuevamente seleccionando primero (b, c) ;
- 3 compruebe que ambas salidas son coberturas;
- 4 encuentre una cobertura óptima;
- 5 calcule las razones observadas;
- 6 explique si la elección cambia la garantía 2.

Pregunta

¿Puede una ejecución producir el óptimo y otra alcanzar la razón 2 sobre el mismo grafo?

Actividad 2: PTAS o FPTAS

Clasifique los siguientes tiempos, suponiendo que todos producen una solución con calidad $(1 + \varepsilon)$:

- ❶ $T(n, \varepsilon) = n^{3/\varepsilon}$.
- ❷ $T(n, \varepsilon) = n^5/\varepsilon^2$.
- ❸ $T(n, \varepsilon) = 3^{1/\varepsilon} n^2$.
- ❹ $T(n, \varepsilon) = 2^n/\varepsilon$.

Justificación requerida

No basta escribir la clasificación. Debe indicarse la dependencia respecto de n y de $1/\varepsilon$.

Mapa de las técnicas estudiadas

Técnica	Qué ofrece	Ejemplo
Factor constante	Garantía fija	Vertex Cover: factor 2
Redondeo	Menos estados y error controlado	Escalamiento de valores
PTAS	Calidad arbitraria para ε fijo	Tiempo como $n^{1/\varepsilon}$
FPTAS	Calidad arbitraria y dependencia polinomial en $1/\varepsilon$	Knapsack 0/1
Cotas	Referencia respecto del óptimo	Matching, relajaciones

Cierre: cinco ideas esenciales

- 1 El redondeo reduce complejidad a cambio de un error que debe acotarse.
- 2 La 2-aproximación de Vertex Cover utiliza los extremos de un matching maximal.
- 3 La desigualdad $|M| \leq \text{OPT}$ proporciona la cota inferior de la demostración.
- 4 Un PTAS es polinomial en n para cada ε fijo.
- 5 Un FPTAS es polinomial tanto en n como en $1/\varepsilon$.

Siguiente tema

Montículos de Fibonacci y análisis amortizado.

Responda individualmente:

- 1 ¿Por qué las aristas seleccionadas en Vertex Cover forman un matching?
- 2 ¿Por qué $|M|$ es una cota inferior para la cobertura óptima?
- 3 ¿Cuál es la diferencia esencial entre PTAS y FPTAS?
- 4 ¿Qué ocurre con calidad y tiempo cuando disminuye ε ?
- 5 ¿Por qué el análisis del redondeo debe incluir la reconstrucción?

Referencias y recursos

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.^a ed. MIT Press, 2022.
-  V. V. Vazirani. *Approximation Algorithms*. Springer, 2001.
-  D. P. Williamson y D. B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011.
-  MIT OpenCourseWare. *Lecture 17: Approximation Algorithms*.
<https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/>
-  MIT OpenCourseWare. *Lecture 18: Polynomial-Time Approximations*.
<https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2012/>